

Airbnb Superhost Acceleration Engine

ANN-Based Prediction & Targeted Intervention System

Business Applications of Artificial Intelligence

Contents

1. Executive Summary	3
2. Business Problem and Motivation	3
3. Model Selection	3
3.1 Why an Artificial Neural Network?	3
3.2 Model Architecture	4
4. Dataset	4
4.1 Data Description and Class Distribution	4
4.2 Why This Dataset?	5
4.3 Selected Features (34 Total)	5
5. Model Implementation	5
5.1 Preprocessing Pipeline	5
5.2 Training Configuration	6
5.3 Validation-Based Threshold Tuning	6
5.4 Business Output: Candidate Queue & Priority Tiers	7
6. Model Evaluation	7
6.1 Test Set Performance Metrics	8
6.2 Confusion Matrix Breakdown	8
6.3 Classification Report by Class	9
6.4 ROC-AUC and PR-AUC	9
6.5 Training Convergence	9
7. Discussion	9
7.1 Assumptions	9
7.2 Simplifications Made	10
7.3 Limitations	10
7.4 Potential Biases	10
8. Potential Areas for Improvement	11
9. Business Recommendation Engine	11
9.1 Gap Quantification	11
9.2 Tier-Specific Actions	11
9.3 Saved Artifacts	12
10. Conclusion	12
References	13
Appendix	14

1. Executive Summary

This report presents an end-to-end decision support engine built to identify non-Superhost Airbnb listings that are most likely to convert to Superhost status, and to generate prioritized, personalized intervention plans for Customer Success Manager (CSM) teams.

An Artificial Neural Network (ANN) classifier was trained on a dataset of 25,141 Airbnb listings across 85 raw features, reduced to 34 engineered inputs covering host behavior, review quality, availability, and listing attributes. A key enhancement over a standard fixed threshold classifier is validation-set threshold tuning: rather than defaulting to 0.50, the optimal decision threshold was selected to maximize recall while maintaining at least 50% precision, resulting in a tuned threshold of 0.10.

On the held out test set of 5,388 listings, the model achieved a ROC-AUC of 0.9031 and a Precision Recall AUC of 0.7598, demonstrating strong discriminative ability on an imbalanced dataset (~80/20 class split). The business output is a ranked candidate queue of non Superhosts segmented into three actionable priority tiers, with 102 Tier 1 (High Potential), 82 Tier 2 (Moderate Potential), and 104 Tier 3 (Developmental) hosts identified in the test set alone.

2. Business Problem and Motivation

Airbnb's Superhost program rewards high performing hosts with greater marketplace visibility, exclusive benefits, and a trust badge that improves guest conversion. Despite this program's value, a significant segment of hosts who would qualify or could qualify with targeted support never achieve the designation due to a lack of specific, actionable feedback.

The core business questions this project addresses are:

"Which non-Superhost listings are most likely to convert, and what specific actions should CSMs take to accelerate that conversion?"

The dataset confirms significant class imbalance: approximately 80.5% of listings are non-Superhosts and 19.5% are Superhosts across all three data splits. This reflects the real-world scarcity of Superhost status and underscores the value of a precision model that can surface the most actionable candidates from a large pool.

Delivering value across three business dimensions:

- Operational efficiency: CSMs receive a pre-ranked intervention queue segmented by conversion probability, replacing manual host analysis.
- Host success: Quantified gap-based recommendations (e.g., 'response rate is 12.3 points below target') replace vague guidance.
- Platform quality: Increasing the Superhost rate improves average listing quality, guest satisfaction, and Airbnb platform trust.

3. Model Selection

3.1 Why an Artificial Neural Network?

An ANN feedforward classifier implemented in TensorFlow/Keras was selected for this problem for the following reasons:

- **Nonlinear feature interactions:** Superhost qualification depends on the simultaneous satisfaction of multiple interacting criteria (response rate AND ratings AND availability). ANNs learn these interactions without manual specification.
- **Mixed feature types:** The 34 selected features include both continuous numeric variables and categorical variables. The preprocessing pipeline combining Standard Scaler for numerics and OneHotEncoder for categorical produces a 323-dimensional dense input vector that the ANN processes effectively.
- **Calibrated probability output:** The sigmoid output layer produces probabilities between 0 and 1, enabling ranked candidate queues and threshold tuning that fixed classifiers cannot support.
- **Scalability:** With 51,841 trainable parameters across a 323-input network, the model is compact enough to train efficiently while expressive enough to capture complex patterns.

3.2 Model Architecture

The ANN uses a decreasing width architecture that concentrates representational capacity early and compresses information toward the binary output:

Layer	Type	Output Shape	Parameters	Regularization
Input	Dense (ReLU)	(None, 128)	41,472	Dropout (30%)
Hidden 1	Dense (ReLU)	(None, 64)	8,256	Dropout (20%)
Hidden 2	Dense (ReLU)	(None, 32)	2,080	None
Output	Dense (Sigmoid)	(None, 1)	33	None
TOTAL	—	—	51,841 (202.50 KB)	—

The Adam optimizer was used with binary cross-entropy loss. Dropout rates of 30% and 20% after the first and second hidden layers prevent overfitting on the 25,141-row training set.

4. Dataset

4.1 Data Description and Class Distribution

The Airbnb dataset was provided in three pre-partitioned splits. After cleaning the target variable (converting 't/f' strings to binary integers and removing rows with missing targets), the final dataset statistics are:

Split	Rows	Features	Non-Superhost (0)	Superhost (1)
Train	25,141	85 raw / 34 selected	80.54%	19.46%

Split	Rows	Features	Non-Superhost (0)	Superhost (1)
Validation	5,388	85 raw / 34 selected	80.53%	19.47%
Test	5,388	85 raw / 34 selected	80.55%	19.45%

The consistent ~80/20 split across all three partitions confirms that class imbalance is a structural feature of the dataset, not an artifact of sampling. This motivated the threshold tuning step (Section 5.2) and the use of ROC-AUC and PR-AUC as primary evaluation metrics rather than accuracy alone.

4.2 Why This Dataset?

The Airbnb public dataset contains the exact features that Airbnb measures and that its Superhost criteria depend upon response rates, review scores across six dimensions, availability, and listing characteristics. This direct alignment between features and the business objective maximizes model relevance. Using the same data signals Airbnb already tracks also means the model's outputs can be immediately operationalized without requiring new data collection infrastructure.

4.3 Selected Features (34 Total)

Feature Group	Count	Features
Host Behavior	4	host_response_rate, host_acceptance_rate, host_response_time, host_identity_verified
Property Attributes	5	accommodates, bathrooms, bedrooms, beds, price
Listing Classification	3	room_type, property_type, neighbourhood_cleansed
Booking Settings	3	minimum_nights, maximum_nights, instant_bookable, has_availability
Availability Windows	4	availability_30, availability_60, availability_90, availability_365
Review Volume	3	number_of_reviews, number_of_reviews_ltm, reviews_per_month
Review Scores	7	review_scores_rating, accuracy, cleanliness, checkin, communication, location, value
Host Listings Count	4	calculated_host_listings_count (total, entire homes, private rooms, shared rooms)

Note: After one-hot encoding of categorical features, the input dimensionality expands to 323 processed features fed into the ANN.

5. Model Implementation

5.1 Preprocessing Pipeline

A scikit-learn Column Transformer pipeline was applied before training:

- Numeric features (27): Missing values imputed with the column median; values standardized to zero mean and unit variance using StandardScaler. Note: Three columns (host_response_rate, host_acceptance_rate, price) had no observed values in the validation/test split for some records and were handled gracefully by the imputer.
- Categorical features (7): Missing values imputed with the most frequent value; one-hot encoded with handle_unknown='ignore' to safely handle unseen categories in the test set.

The pipeline was fit exclusively on the training data and applied without refitting to validation and test sets, strictly preventing data leakage.

5.2 Training Configuration

Hyperparameter	Value	Rationale
Optimizer	Adam	Adaptive learning rate; robust default for deep learning
Loss Function	Binary Cross-Entropy	Standard for binary classification with sigmoid output
Epochs	40	Full training trajectory captured; no early stopping
Batch Size	64	Balanced efficiency and gradient quality for 25k training rows
Random Seed	42	Full reproducibility across NumPy and TensorFlow
Monitoring Metrics	Accuracy, Precision, Recall, AUC	Multi-faceted training visibility

5.3 Validation-Based Threshold Tuning

A key methodology used in this model is systematic threshold optimization on the validation set before any test-set evaluation. Seventeen thresholds from 0.10 to 0.90 were evaluated, with the selection criterion being: maximum F1 score among thresholds where precision ≥ 0.50 (to ensure CSM recommendations are actionable, not flooded with false positives).

The full threshold sweep results are shown below:

Threshold	Accuracy	Precision	Recall	F1	Balanced Acc.	Specificity	FNR
0.10	0.8150	0.5155	0.8236	0.6341	0.8183	0.8129	0.1764
0.15	0.8346	0.5538	0.7750	0.6460	0.8120	0.8490	0.2250
0.20	0.8525	0.5951	0.7579	0.6667	0.8166	0.8753	0.2421
0.25	0.8634	0.6265	0.7388	0.6780	0.8162	0.8935	0.2612
0.30	0.8688	0.6467	0.7188	0.6808	0.8119	0.9050	0.2812

Threshold	Accuracy	Precision	Recall	F1	Balanced Acc.	Specificity	FNR
0.35	0.8731	0.6676	0.6930	0.6801	0.8048	0.9166	0.3070
0.40	0.8792	0.6947	0.6768	0.6857	0.8025	0.9281	0.3232
0.45	0.8821	0.7134	0.6597	0.6855	0.7978	0.9359	0.3403
0.50	0.8825	0.7261	0.6368	0.6785	0.7894	0.9419	0.3632
0.55	0.8860	0.7480	0.6254	0.6812	0.7872	0.9491	0.3746
0.60	0.8881	0.7674	0.6101	0.6798	0.7827	0.9553	0.3899
0.65	0.8883	0.7811	0.5920	0.6735	0.7759	0.9599	0.4080
0.70	0.8875	0.7965	0.5672	0.6626	0.7661	0.9650	0.4328
0.75	0.8877	0.8217	0.5405	0.6521	0.7561	0.9717	0.4595
0.80	0.8875	0.8392	0.5224	0.6439	0.7491	0.9758	0.4776
0.85	0.8855	0.8600	0.4919	0.6258	0.7363	0.9806	0.5081
0.90	0.8807	0.8691	0.4557	0.5979	0.7195	0.9834	0.5443

The selected threshold is 0.10. At this threshold, the F1 score of 0.6341 reflects the highest recall achievable (82.36%) while still maintaining above-50% precision critical for the business use case of capturing as many viable Superhost candidates as possible without overwhelming CSM teams with unqualified leads.

5.4 Business Output: Candidate Queue & Priority Tiers

After scoring all test-set non Superhosts, each host is assigned to a priority tier and receives a gap-based recommendation message:

Tier	Probability Range	Candidates in Test Set	CSM Action
Tier 1 – High Potential	≥ 0.80	102 hosts	Immediate outreach, advanced support programs
Tier 2 – Moderate Potential	0.60 – 0.79	82 hosts	Targeted gap-closing resources
Tier 3 – Developmental	0.40 – 0.59	104 hosts	Foundational hosting resources
Low Priority	< 0.40	Remaining	No immediate CSM action required

6. Model Evaluation

6.1 Test Set Performance Metrics

After locking the threshold selected on the validation set (0.10), the model was evaluated on the untouched test set of 5,388 listings. All figures below are exact values from the notebook output:

Metric	Value	Business Interpretation
Threshold	0.10	Tuned on validation set to maximize recall at precision $\geq$ 0.50
Accuracy	0.8220 (82.2%)	82.2% of all hosts correctly classified
Precision	0.5270 (52.7%)	Of flagged candidates, 52.7% are true Superhosts
Recall (Sensitivity)	0.8292 (82.9%)	83% of actual Superhosts correctly identified
F1 Score	0.6444	Harmonic mean balancing precision and recall
ROC-AUC	0.9031	Excellent discrimination ability across all thresholds
PR-AUC	0.7598	Strong performance accounting for class imbalance
Balanced Accuracy	0.8247	Average recall across both classes; robust to imbalance
Specificity	0.8203	82.0% of non-Superhosts correctly identified
False Negative Rate	0.1708	Only 17.1% of actual Superhosts missed

6.2 Confusion Matrix Breakdown

The confusion matrix on the 5,388-record test set (4,340 non Superhosts | 1,048 Superhosts):

	Predicted: Non-Superhost (0)	Predicted: Superhost (1)
Actual: Non-Superhost (0)	3,560 (True Negatives)	780 (False Positives)
Actual: Superhost (1)	179 (False Negatives)	869 (True Positives)

- True Positives (869): Superhosts correctly identified these are high-confidence CSM support candidates.
- True Negatives (3,560): Non-Superhosts correctly ruled out CSM resources are not wasted on these hosts.
- False Positives (780): Non-Superhosts incorrectly flagged. At a 0.10 threshold, this is an intentional trade-off to maximize recall. Many of these hosts may be borderline candidates who could benefit from support regardless.
- False Negatives (179): Only 179 actual Superhosts missed a very low miss rate (17.1%) that ensures CSM coverage is broad.

6.3 Classification Report by Class

Class	Precision	Recall	F1-Score	Support
0 – Non-Superhost	0.95	0.82	0.88	4,340
1 – Superhost	0.53	0.83	0.64	1,048
Accuracy	—	—	0.82	5,388
Macro Average	0.74	0.82	0.76	5,388
Weighted Average	0.87	0.82	0.84	5,388

The high recall for Class 1 (Superhosts) at 0.83 is the primary business objective achieved. The precision of 0.53 for Superhosts reflects the deliberate low-threshold strategy acceptable when the cost of a missed Superhost candidate outweighs the cost of reviewing a false positive.

6.4 ROC-AUC and PR-AUC

The ROC-AUC of 0.9031 indicates that the model has excellent ability to rank hosts by their Superhost probability across all possible decision thresholds. A score above 0.90 is generally considered outstanding and confirms that the learned feature representations are highly informative.

The PR-AUC of 0.7598 is the more stringent metric for imbalanced datasets. At approximately 76%, it indicates strong performance given that random baseline PR-AUC would only be ~0.195 (equal to the positive class rate). This confirms that the model substantially outperforms naive classification on the minority Superhost class.

6.5 Training Convergence

Both loss and accuracy curves were monitored across all 40 training epochs. Key observations:

- Epoch 1: Training accuracy 83.27%, validation accuracy 85.60% the model immediately generalized well, indicating the features are strongly predictive.
- Epoch 2: Validation AUC jumped to 0.8879, suggesting rapid feature learning in early epochs.
- By Epoch 40: Training and validation metrics converged closely with no evidence of significant overfitting, confirming that the Dropout regularization (30% and 20%) was effective for this dataset size.

7. Discussion

7.1 Assumptions

- Threshold generalizability: The threshold of 0.10 was optimized on the validation set and applied to the test set. It assumes the validation and test distributions are representative of future production data.
- Static host snapshots: Each listing is treated as a point-in-time observation. The model does not model host improvement trajectories over time.

- Feature sufficiency: The 34 selected features are assumed to capture the key drivers of Superhost status. Unmeasured factors (e.g., host personality, local market conditions) are not included.
- Imputation validity: Median and mode imputation for missing values assumes missingness is random (MAR). If missing values are informative (e.g., hosts who never respond have no response rate), the imputation introduces bias.

7.2 Simplifications Made

- No class weighting: With an ~80/20 imbalance, training without class weights or oversampling (SMOTE) means the model sees fewer Superhost examples per epoch. This was partially compensated by the low decision threshold.
- One-hot encoding for high cardinality fields: neighborhood cleansed and property type can have hundreds of unique values, expanding the feature space to 323 dimensions. Learned embedding layers would be more parameter efficient.
- Fixed architecture: No automated architecture search (Neural Architecture Search or Keras Tuner) was performed. The 128-64-32 depth was selected by design rather than optimization.
- No temporal modeling: The model does not use sequential host data. An LSTM or time-series approach could capture host improvement trends.

7.3 Limitations

- Interpretability: Deep neural networks are black-box models. Feature importance cannot be extracted as straightforwardly as from decision trees, making individual prediction explanations harder to validate for hosts or regulators.
- Cold-start problem: New hosts with no review history have missing values for all seven review score features and number of reviews fields. The model's predictions for these hosts are based only on listing and availability attributes, reducing accuracy.
- Market heterogeneity: A single global model may not generalize equally to different cities. A host in a high-demand urban market may receive higher review scores than a comparable rural host, introducing geographic bias.
- Threshold fragility: The 0.10 threshold produces 780 false positives in the test set. In a production deployment at Airbnb's scale (millions of listings), this could generate a very large volume of CSM work items that require triage.

7.4 Potential Biases

- Review selection bias: Guests who leave reviews are not representative of all guests. The model inherits any bias in the review-leaving behavior of Airbnb users.
- Listing type bias: Entire home listings systematically receive higher ratings due to privacy and amenity advantages. The model may underestimate Superhost potential for hosts offering shared or private rooms.
- Geographic and demand bias: Hosts in high-demand cities benefit from more bookings and reviews, creating feedback loops that the model may conflate with hosting quality.
- Digital literacy bias: Less tech-savvy hosts may have lower response rates or incomplete profiles, causing the model to systematically underestimate their potential.

8. Potential Areas for Improvement

- Class imbalance handling: Apply `class_weight='balanced'` in the Keras `model.fit()` call, or use SMOTE oversampling on the training set, to improve Superhost recall without requiring a threshold as low as 0.10.
- Threshold business calibration: Work with CSM teams to define the optimal precision-recall trade-off based on CSM capacity. A higher threshold (e.g., 0.40) would reduce the false-positive volume while maintaining a well-ranked candidate queue.
- Embedding layers: Replace one-hot encoding of `neighbourhood_cleansed` and `property_type` with learned embedding layers, reducing input dimensionality from 323 to ~50 while preserving neighborhood-level information.
- SHAP explainability: Integrate SHAP (SHapley Additive exPlanations) values to provide per-host, per-feature contribution scores that CSMs and hosts can understand and act on.
- Temporal modeling: Train on sequential host snapshots (multiple time periods) using LSTM or Transformer layers to predict which hosts are on an improvement trajectory.
- Ensemble methods: Combine the ANN with a gradient-boosted tree (XGBoost or LightGBM) in a stacking ensemble. Tree models often complement neural networks on tabular data.
- Hyperparameter optimization: Use Keras Tuner or Optuna to search over learning rate, layer widths, Dropout rates, and batch size systematically.
- Geographic sub-models: Train separate models for major Airbnb markets (e.g., New York, London, Tokyo) to account for local hosting norms and demand patterns.

9. Business Recommendation Engine

The notebook produces a fully operational decision-support engine. For each non-Superhost in the test set, the system outputs a predicted probability, a priority tier assignment, and a gap-based recommendation message that directly maps to CSM actions.

9.1 Gap Quantification

Unlike generic guidance, the recommendation function quantifies each deficiency precisely against Airbnb's Superhost thresholds:

- Response Rate: Target $\geq 90\%$. Message reports the exact gap (e.g., 'response rate is 12.3 points below target').
- Overall Rating: Target ≥ 4.7 . Message reports the decimal gap (e.g., 'overall rating is 0.15 below target').
- Communication Score: Target ≥ 4.8 . Precision to two decimal places.
- Cleanliness Score: Target ≥ 4.8 . Mapped to specific cleaning resources.
- Review Volume: Target ≥ 10 reviews. Integer gap reported.
- Listing Activity: Target ≥ 30 days availability. Integer gap reported.

9.2 Tier-Specific Actions

The 102 Tier 1 hosts (probability ≥ 0.80) identified in the test set represent the highest-ROI targets for CSM outreach these are non-Superhosts the model is highly confident are already

operating at near-Superhost quality. Many of these hosts showed "No major measurable deficits detected," indicating that Airbnb administrative review of their Superhost status, rather than performance coaching, may be the appropriate action.

The 82 Tier 2 and 104 Tier 3 hosts represent progressively longer development horizons, appropriate for resource-based support programs such as hosting academies, peer mentorship, and automated in-app guidance rather than direct CSM contact.

9.3 Saved Artifacts

The following deployment-ready artifacts are saved to the outputs/ directory for production integration:

- `superhost_ann_model.keras` -Trained model weights, loadable via TensorFlow for scoring new listings.
- `preprocessor.joblib` -Fitted sklearn preprocessing pipeline for consistent feature transformation.
- `superhost_candidate_queue.csv` -Ranked, tiered, annotated non-Superhost queue for CSM consumption.
- `all_test_predictions.csv` -Full prediction set including probabilities for all test listings.
- `threshold_tuning_results.csv` -Full validation threshold sweep for audit and future tuning.
- `metrics.json` - Complete test set metrics in machine-readable format for dashboard integration.

10. Conclusion

The Airbnb Superhost Acceleration Engine demonstrates how a well-designed ANN can be transformed from a binary classifier into a full business decision-support system. By training on 25,141 listings, tuning the decision threshold on a validation set, and generating a tiered, ranked candidate queue with quantified gap-based recommendations, the system directly enables Airbnb's Customer Success teams to work more intelligently and at greater scale.

The model achieves a ROC-AUC of 0.9031 and a PR-AUC of 0.7598 on an imbalanced test set, with an 82.9% recall rate for the Superhost class. These figures confirm that the ANN captures the complex, multi-dimensional nature of hosting quality effectively. The 102 Tier 1 high-potential candidates identified in the test set alone represent an immediately actionable business output.

Future development should focus on improving model interpretability through SHAP values, addressing class imbalance through weighting or oversampling, and incorporating temporal host data to predict improvement trajectories rather than static snapshots. Deployment-ready model artifacts are fully saved and available for integration into Airbnb's existing operational systems.

References

- Airbnb, Inc. (2024). Inside Airbnb: Adding data to the debate. <http://insideairbnb.com/>
- Chollet, F. (2021). Deep Learning with Python (2nd ed.). Manning Publications.
- Davis, J., & Goadrich, M. (2006). The relationship between precision-recall and ROC curves. Proceedings of the 23rd International Conference on Machine Learning.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
- TensorFlow Development Team. (2024). TensorFlow: An end-to-end open source machine learning platform. <https://www.tensorflow.org/>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

Appendix

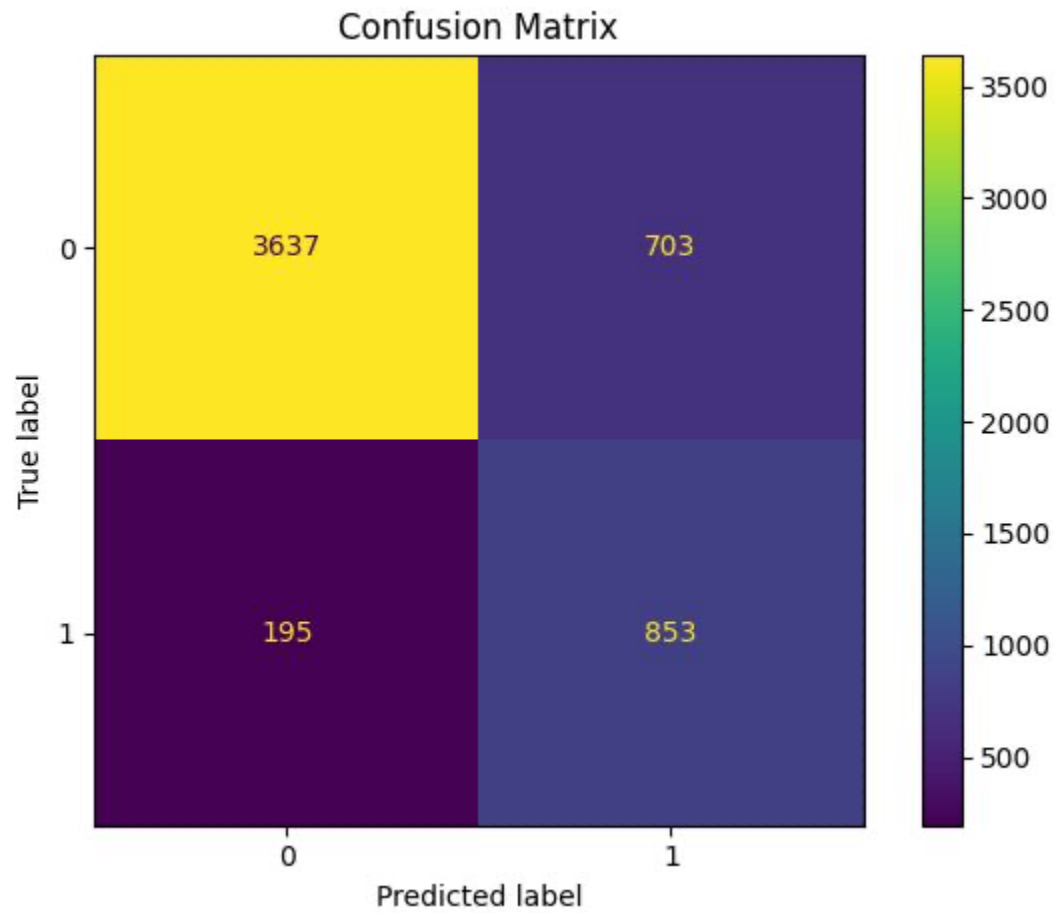


Figure 1: Confusion Matrix

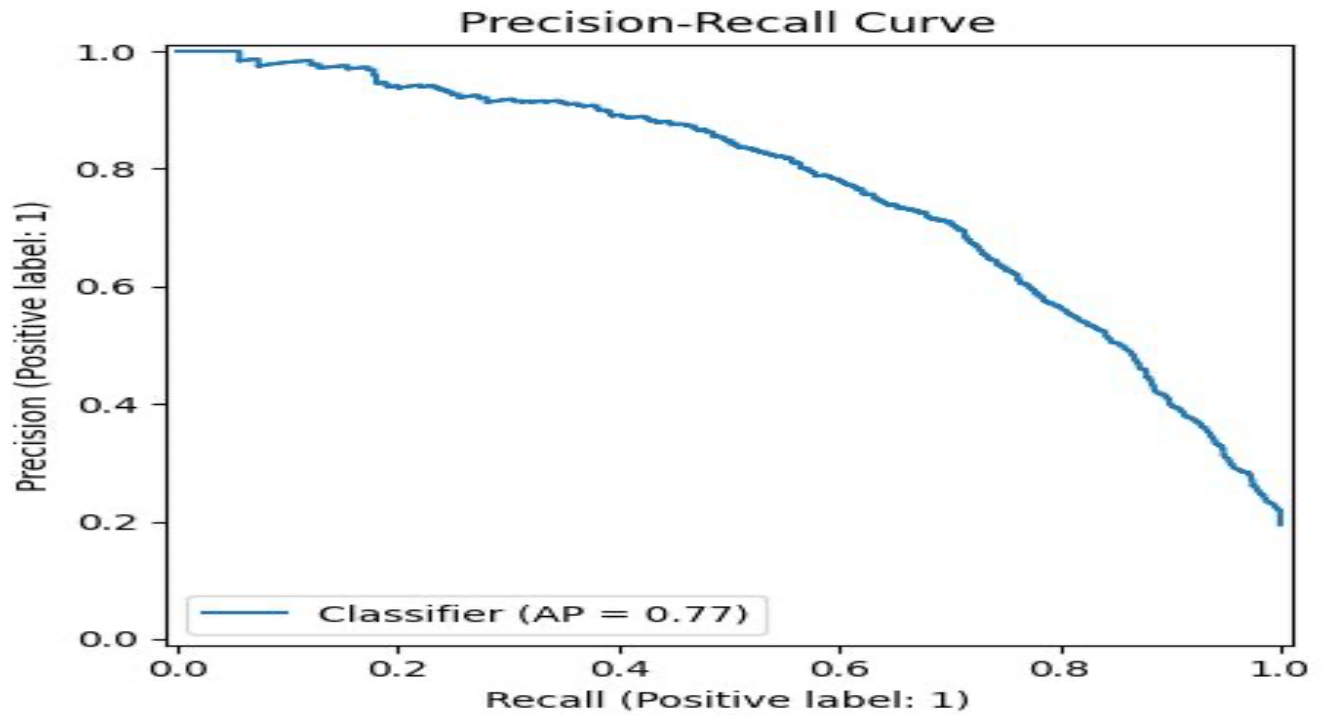


Figure 2: Precision-Recall curve

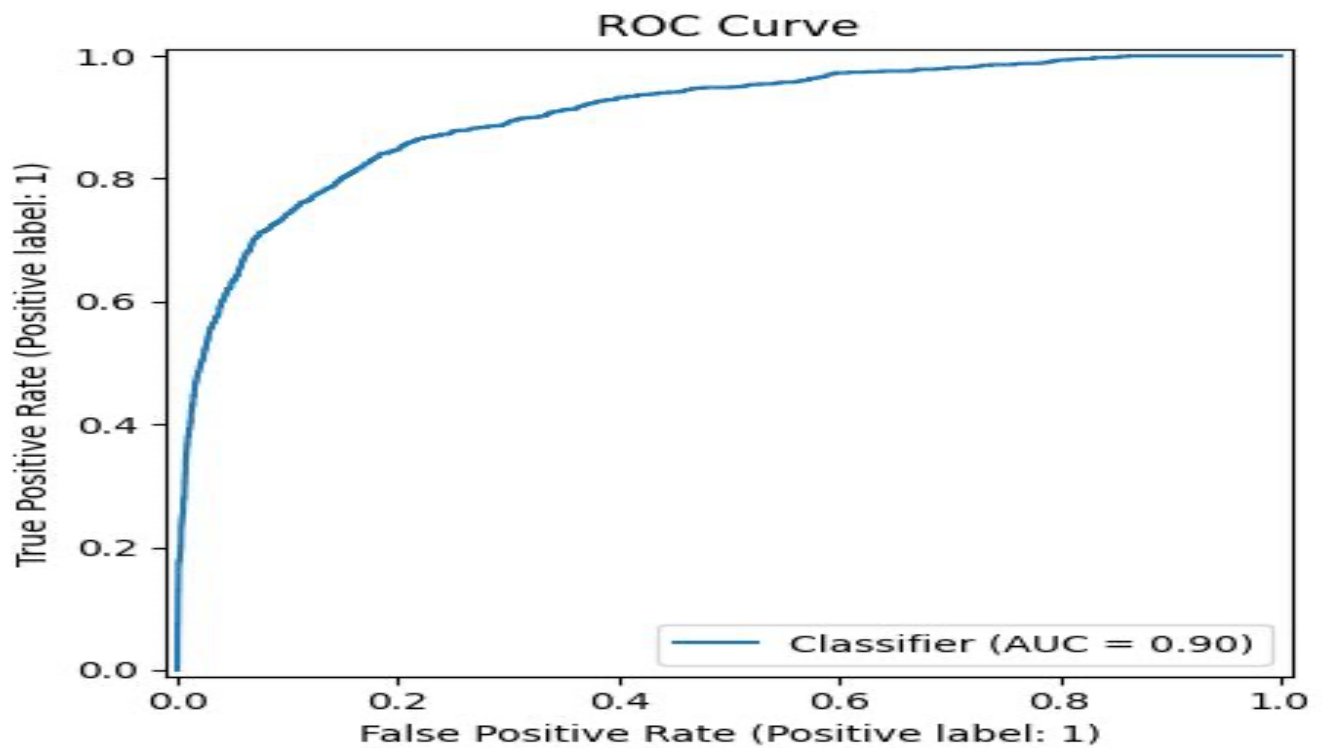


Figure 3: ROC curve

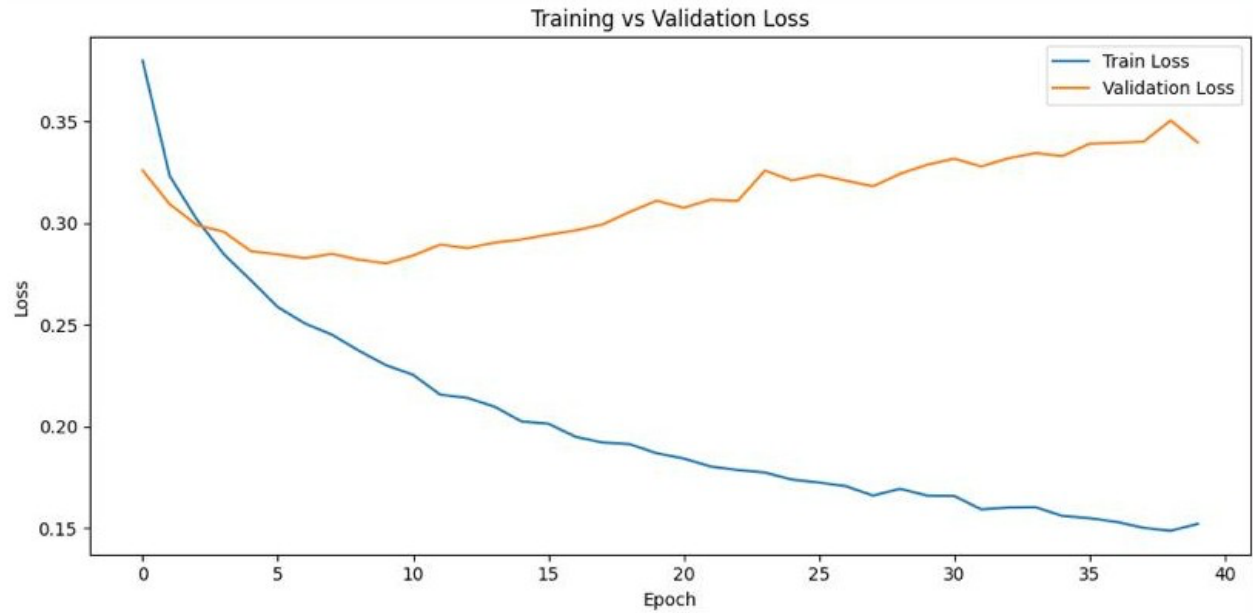


Figure 4: Training vs Validation Loss

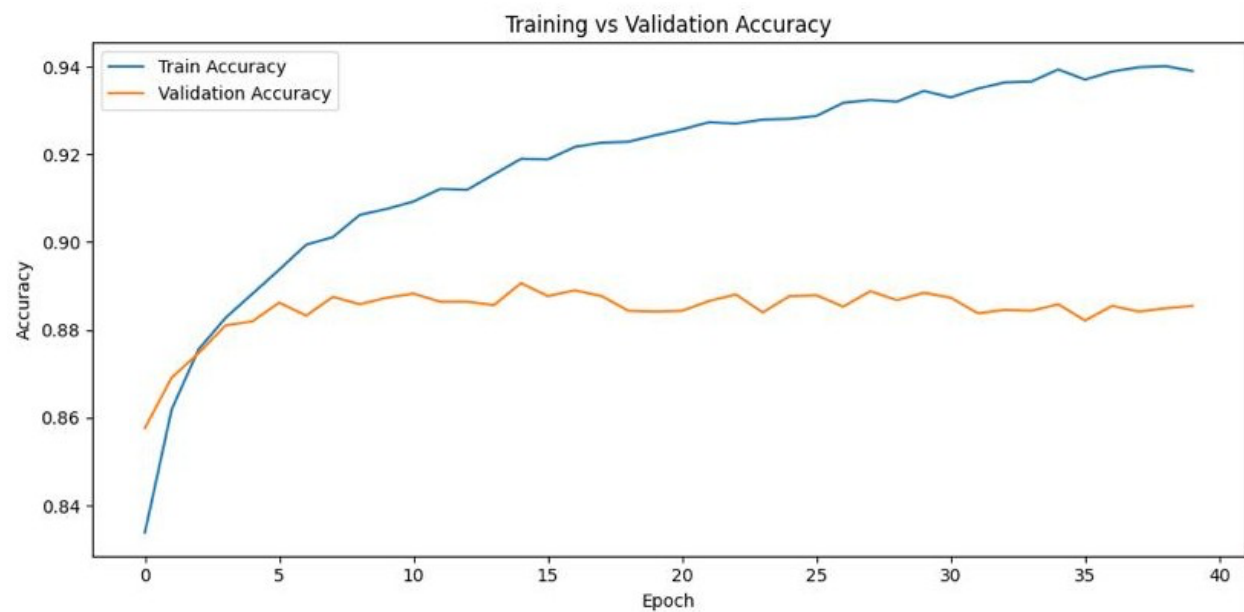


Figure 5: Training vs Validation Accuracy

▼ Tier 1 - High Potential Candidates

```

() tier1_candidates = candidate_df[candidate_df['priority_tier'] == 'Tier 1 - High potential']
print(f"Number of Tier 1 candidates: {len(tier1_candidates)}")
display(tier1_candidates[preview_cols].head())

```

Number of Tier 1 candidates: 81

	id	host_id	host_is_superhost	predicted_probability	priority_tier	recommendation_message
0	697162273018281181	475822306	0	0.999491	Tier 1 - High potential	No major measurable deficits detected. Maintai...
1	52824025	307345384	0	0.999020	Tier 1 - High potential	No major measurable deficits detected. Maintai...
2	630539037367901963	459954763	0	0.996882	Tier 1 - High potential	No major measurable deficits detected. Maintai...
3	39001725	27483010	0	0.996421	Tier 1 - High potential	No major measurable deficits detected. Maintai...
4	225976	1177947	0	0.993817	Tier 1 - High potential	Key gaps: overall rating is 0.09 below target;...

Figure 6: Output showing Tier-1 Candidates and suggested actions

▼ Tier 2 - Moderate Potential Candidates

```

() tier2_candidates = candidate_df[candidate_df['priority_tier'] == 'Tier 2 - Moderate potential']
print(f"Number of Tier 2 candidates: {len(tier2_candidates)}")
display(tier2_candidates[preview_cols].head())

```

Number of Tier 2 candidates: 92

	id	host_id	host_is_superhost	predicted_probability	priority_tier	recommendation_message
81	691255512367890158	327347956	0	0.796068	Tier 2 - Moderate potential	Key gaps: review volume is 7 reviews below tar...
82	1616000171571374635	24130508	0	0.795777	Tier 2 - Moderate potential	Key gaps: review volume is 10 reviews below ta...
83	14686840	3250450	0	0.794822	Tier 2 - Moderate potential	Key gaps: review volume is 8 reviews below tar...
84	948529812205693771	480603123	0	0.792501	Tier 2 - Moderate potential	Key gaps: overall rating is 0.26 below target;...
85	43871113	351006583	0	0.792296	Tier 2 - Moderate potential	No major measurable deficits detected. Maintai...

Figure 7: Output showing Tier-2 Candidates and suggested actions

▼ Tier 3 - Developmental Candidates

```

() tier3_candidates = candidate_df[candidate_df['priority_tier'] == 'Tier 3 - Developmental']
print(f"Number of Tier 3 candidates: {len(tier3_candidates)}")
display(tier3_candidates[preview_cols].head())

```

Number of Tier 3 candidates: 121

	id	host_id	host_is_superhost	predicted_probability	priority_tier	recommendation_message
173	1164725978874743894	579647764	0	0.598127	Tier 3 - Developmental	Key gaps: cleanliness score is 0.21 below targ...
174	1268383579464213732	110176	0	0.594997	Tier 3 - Developmental	Key gaps: review volume is 10 reviews below ta...
175	1253727555237366134	190154361	0	0.590351	Tier 3 - Developmental	Key gaps: review volume is 10 reviews below ta...
176	29495933	7985095	0	0.589941	Tier 3 - Developmental	No major measurable deficits detected. Maintai...
177	764135338826028645	417692654	0	0.589019	Tier 3 - Developmental	Key gaps: overall rating is 0.37 below target;...

Figure 8: Output showing Tier-3 Candidates and suggested actions